```
"""
nexus.core.di
===============

A deliberately small dependency-
injection container.

NEXUS does not use a heavyweight DI
framework (e.g. ``dependency-
injector``)
because the composition root is
simple enough that a ~150-line
container
covering the actual needs
(singletons, factories, interface-
keyed lookup)
is easier to audit than a general-
purpose framework, and keeps the core
package dependency-free.

Two registration styles are
supported:
    * ``register_instance``  — an
already-constructed singleton.
    * ``register_factory``   — a
callable invoked lazily on first
```

```
      ``resolve()``; the result is
cached (singleton-by-default) unless
      ``singleton=False`` is passed,
in which case a new instance is built
      on every ``resolve()`` call.

Components are looked up by
*interface type* (typically one of
the
Protocols in
``nexus.core.interfaces``), not by
string name, so
``mypy``/IDE tooling can verify a
resolved object satisfies the
interface
the caller asked for.
"""

from __future__ import annotations

import threading
from typing import Callable, TypeVar

from nexus.core.exceptions import
ProviderNotRegisteredError

T = TypeVar("T")
```

```
class Container:
    """
    Thread-safe registry mapping an
interface type to a concrete
provider.

    Example:
        container = Container()

container.register_instance(NexusConf
ig, my_config)

container.register_factory(LLMProvide
r, lambda:
ClaudeProvider(api_key=...))

        llm =
container.resolve(LLMProvider)  # ->
ClaudeProvider instance
    """

    def __init__(self) -> None:
        self._instances: dict[type,
object] = {}
        self._factories: dict[type,
Callable[[], object]] = {}
        self._singleton_flags:
dict[type, bool] = {}
        self._lock =
```

```
threading.RLock()

    def register_instance(self,
interface: type[T], instance: T) ->
None:
        """Register an already-
constructed object as the provider
for ``interface``."""
        with self._lock:

self._instances[interface] = instance

    def register_factory(
        self, interface: type[T],
factory: Callable[[], T], *,
singleton: bool = True
    ) -> None:
        """
        Register a factory callable
for ``interface``.

        Args:
            factory: Zero-argument
callable that constructs the
provider.
                Kept zero-argument
deliberately — any dependencies the
                factory needs should
be captured via closure from other
```

```
                already-resolved
container entries at registration
time,
                keeping the container
itself free of a complex "auto-wire
                constructor arguments
by type" mechanism that would be
                harder to reason
about and debug.
            singleton: If True
(default), the factory runs once and
the
                result is cached for
subsequent ``resolve()`` calls. If
                False, a fresh
instance is constructed on every
resolve —
                useful for stateful,
non-shareable objects (e.g. a fresh
                Appium session per
task).
        """
        with self._lock:

self._factories[interface] = factory

self._singleton_flags[interface] =
singleton
```

```
    def resolve(self, interface:
type[T]) -> T:
        """
        Return the provider
registered for ``interface``.

        Raises:

ProviderNotRegisteredError: if
nothing was registered for this
                interface. This is a
fail-fast design choice — NEXUS
should
                never silently
proceed with a missing dependency
(e.g.
                silently skipping the
guardrail engine because it wasn't
                wired up would be a
serious safety bug).
        """
        with self._lock:
            if interface in
self._instances:
                return
self._instances[interface]  # type:
ignore[return-value]

            if interface in
```

```
self._factories:
                singleton =
self._singleton_flags.get(interface,
True)
                if singleton:
                    instance =
self._factories[interface]()

self._instances[interface] = instance
                    return instance
# type: ignore[return-value]
                return
self._factories[interface]()  # type:
ignore[return-value]

        raise
ProviderNotRegisteredError(
            f"No provider registered
for interface {interface!r}. "
            "Register one with
container.register_instance() or "

"container.register_factory() before
resolving it.",
            details={"interface":
getattr(interface, "__name__",
str(interface))},
        )
```

```
    def is_registered(self,
interface: type) -> bool:
        """Check registration without
raising — useful for optional
dependencies."""
        with self._lock:
            return interface in
self._instances or interface in
self._factories

    def clear(self) -> None:
        """
        Remove all registrations.
Primarily intended for test teardown,
so
        one test's container state
can never leak into another test.
        """
        with self._lock:
            self._instances.clear()
            self._factories.clear()

self._singleton_flags.clear()
```